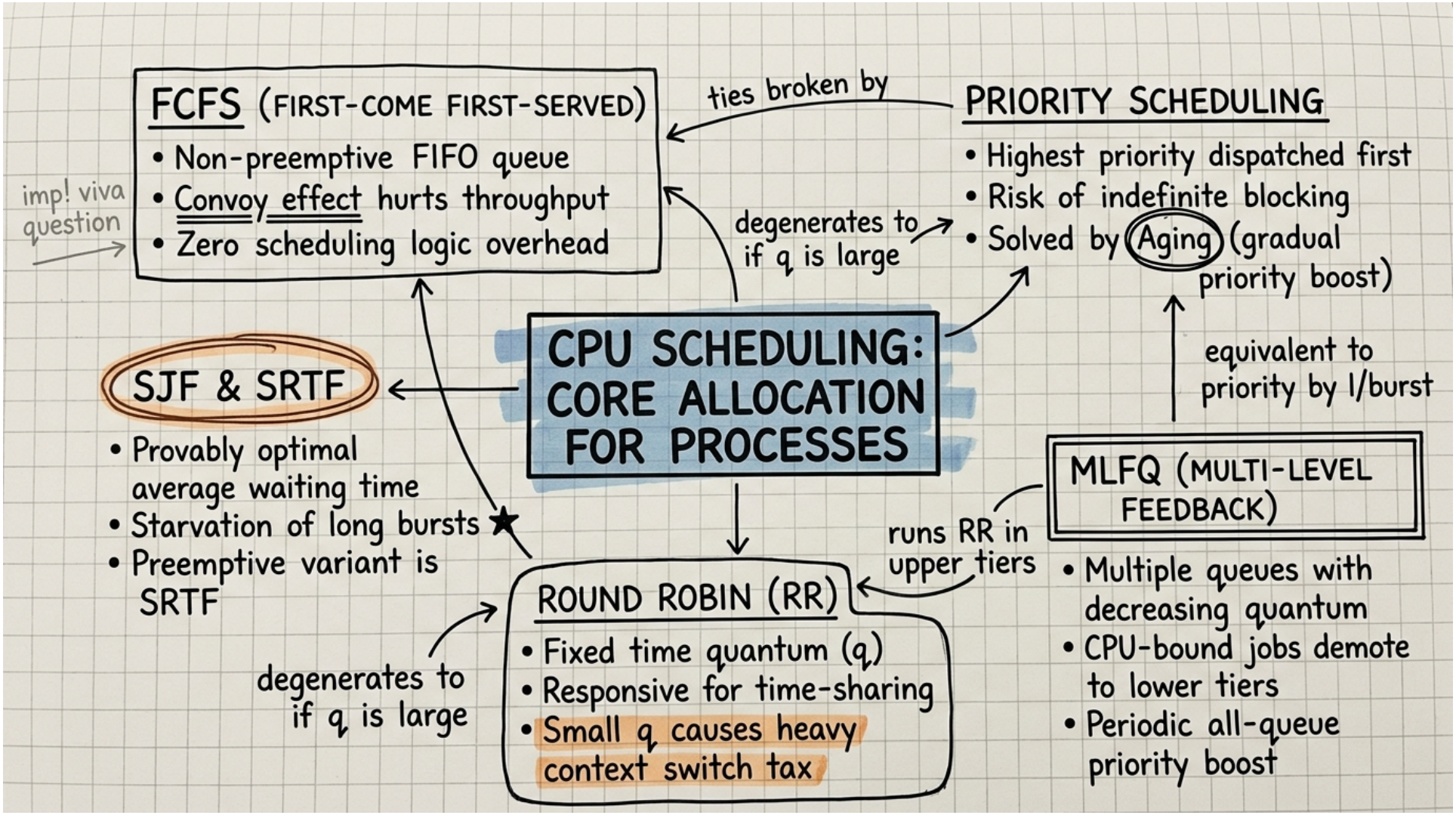
CPU SCHEDULING ALGORITHMS - REVISION CONCEPT CHART

3rd-Year B.Tech Computer Science & Engineering - OS Viva & Exam Revision Guide



1. ALGORITHM COMPARISON MATRIX

Algorithm	Preemptive?	Dispatch Criteria	Optimal Advantage	Primary Pitfall / Fix
FCFS	No	Earliest arrival in FIFO queue	Simple, 0 context-switch overhead	**Convoy Effect:** short jobs blocked by long CPU burst
SJF / SRTF	SJF: No SRTF: Yes	Minimum burst / remaining time	**Provably minimal avg WT**	**Starvation** of long jobs; requires burst prediction
Round Robin (RR)	Yes	FIFO rotation with time slice q	High responsiveness for interactive tasks	Small q creates heavy **context-switch tax**
Priority	Both supported	Highest rank (lowest integer value)	Enforces mission-critical execution	Starvation of low priority; Solved by **Aging**
MLFQ	Yes	Top priority tier down to bottom	Adapts automatically without advance burst knowledge	Parameter tuning sensitivity; Solved by **Periodic Boost**

2. CORE PERFORMANCE METRICS & FORMULAS

- **Turnaround Time (TAT)** = Completion Time - Arrival Time | *Total time spent from submission to termination.*
- **Waiting Time (WT)** = Turnaround Time - Burst Time | *Total time spent idle inside the ready queue.*
- **Response Time (RT)** = First Execution Time - Arrival Time | *Crucial for time-sharing interactive operating systems.*
- **Exponential Burst Smoothing:** $\tau_{n+1} = \alpha \cdot t_n + (1 - \alpha) \cdot \tau_n$ | *Predicts next burst using historical weighted averages (α usually 0.5).*
- **Context Switch Rule of Thumb:** Set time quantum q such that 80% of CPU bursts are shorter than q.

3. HIGH-YIELD VIVA QUESTIONS

Q1: Why can't general-purpose operating systems directly implement pure SJF?
Ans: Pure SJF requires exact advance knowledge of future CPU burst lengths, which is undecidable in general-purpose computing. Systems use exponential smoothing or MLFQ to approximate it dynamically.

Q2: What is the Convoy Effect in FCFS and how does Round Robin eliminate it?
Ans: The Convoy Effect occurs when a CPU-bound process with a huge burst monopolizes the processor, forcing multiple short I/O-bound jobs to wait. Round Robin pre-empts the heavy job after time q, allowing short jobs to run and proceed to I/O devices.

Q3: How does Priority Aging prevent starvation?
Ans: Aging gradually increments the priority of processes that have waited in the ready queue for a defined interval (e.g. +1 priority rank every k ticks). Eventually, even the lowest priority task reaches the highest rank and executes.